Project Report

on

Rural Chatbot

in

Submitted as a partial fulfilment oft he requirements for the award of the degree

of

Bachelor of Technology

in

Computer Science & Engineering[CSE /AIML]

SubmittedBy

Name: Nitish Kumar ***Roll No. 2202201530032***

Name: Ashu ***Roll No. 2202200100033***

Name : Homesh Pal ***Roll No. 2202200100054***

Under the Guidance of

Asst. Prof. - Vaibhav Tiwari

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Hi-Tech Institute of Engineering and Technology,

Ghaziabad [U.P.]

Approved by AICTE and affiliated to Dr. A. P. J. Abdul Kalam Technical University

Lucknow, Uttar Pradesh

Session – 2025 – 2026

CANDIDATE'S DECLARATION

We, **Nitish Kumar, Ashu, Homesh Pal,** hereby affirm that this submission is our original and independent work. To the best of our knowledge and belief, it does not contain any material previously published by another individual, nor does it include content that has been substantially accepted for the award of another degree or diploma by any university or other institute of higher learning, except where proper acknowledgment has been provided within the text. Furthermore, we confirm that this work has been carried out with academic integrity, adhering to the ethical guidelines and standards of scholarly research. Any sources, references, or contributions from external entities have been appropriately cited and acknowledged. We take full responsibility for the accuracy and authenticity of the content herein and affirm that no portion of this submission has been fabricated, plagiarized, or misrepresented. By making this declaration, we acknowledge our commitment to academic honesty and pledge that this work reflects our genuine effort and intellectual contribution.

Group Members Name

1. Nitish Kumar 2202201530032

2. Ashu 2202200100033

3. Homesh Pal 2202200100054

CERTIFICATE

This is to certify that the project report titled ".**RURAL CHATBOT** " submitted by **Nitish Kumar Roll. No. .2202201530032, Ashu Roll. No.2202200100033 and Homesh Pal Roll. No.2202200100054** in partial fulfilment of the requirements for the degree of Bachelor of Technology in the Department of Computer Science and Engineering at Hi Tech Institute of Engineering and Technology, Ghaziabad (U.P.), affiliated with Dr. A.P.J. Abdul Kalam Technical University, Uttar Pradesh, Lucknow (approved by AICTE), is a record of the candidate's original work conducted under my supervision. The research, analysis, and implementation carried out in this project demonstrate the candidate's commitment to academic excellence and technical proficiency. The work presented showcases innovative thinking, problem-solving abilities, and adherence to ethical research practices. The content presented in this project is authentic, and to the best of my knowledge, it has not been submitted for the award of any other degree. Furthermore, this project serves as a significant contribution to the field of computer science and engineering, reflecting the candidate's dedication to advancing knowledge and technical capabilities.

Issued under my supervision.

Date:

Signature of Supervisor ***Signature of HOD***

ACKNOWLEDGEMENT

With profound gratitude, we extend our heartfelt appreciation to our esteemed guide and supervisor, **Asst. Prof. Vaibhav Tiwari**, for his invaluable guidance in planning and executing this project in a timely manner. His unwavering confidence and dynamic leadership have been instrumental in shaping our work. His support during challenging moments will be cherished always, and his insightful recommendations throughout the course of this endeavour are deeply appreciated. We would also like to express our sincere thanks to **Shri Anand Prakash (Chairman), Dr. Pankaj Mishra (Director),** and **Dr. Tripti Chaudhary (Head of Department, Department of Computer Science & Engineering)** at **Hi-Tech Institute of Engineering and Technology, Ghaziabad (U.P.)**, for their timely assistance in facilitating our work. The cooperation and encouragement extended by the faculty members of our department are gratefully acknowledged. Furthermore, we recognize the contributions of the laboratory staff and administrative staff of the department for their support and assistance.

Date:

Group Members Name
1. Nitish Kumar 2202201530032
2. Ashu 2202200100033
3. Homesh Pal 2202200100054

ABSTRACT

Rural India is home to over 65% of the country's population, yet this vast demographic remains largely disconnected from the benefits of the hundreds of government welfare schemes launched for their upliftment. The primary barriers are language, literacy, complexity of digital portals, and dependence on intermediaries who often distort or gatekeep information. The **Rural Chatbot** is an AI-powered, multilingual, voice-enabled web application that directly addresses this crisis. It provides a conversational interface through which rural citizens can ask questions about government schemes in their own language or dialect — by voice or text — and receive personalized, easy-to-understand answers instantly. The system is built on a modern, scalable technology stack: React.js for the frontend, Node.js and Express.js for the backend, MongoDB for flexible data storage, and the Google Gemini NLP API for natural language understanding across Indian languages and dialects. A profile-based eligibility engine filters and ranks schemes based on user attributes such as age, income, occupation, category (SC/ST/OBC/General), and state, delivering personalized recommendations rather than generic lists. Voice input and text-to-speech output (via the Web Speech API) make the system accessible even to users with limited literacy. The interface is deliberately minimal — large buttons, simple navigation, and local language labels — to work for users with little prior digital experience. Performance is ensured through rate limiting, optimized database queries, and a service-based backend architecture following SOLID principles. The system is designed to run on low-end Android smartphones with basic 4G connectivity, ensuring it is practical for rural deployment. Future enhancements include standalone kiosk deployment in panchayat offices, direct integration with official government APIs, WhatsApp and SMS channel support, and coverage of 10+ Indian scheduled languages. The Rural Chatbot aligns with the Digital India mission and demonstrates that AI can be a powerful and equitable force for social development

GITHUBREPOSITORY: https://github.com/nitishprajapati99/RuralChatbot

Repository Contents

Frontend (React): https://github.com/nitishprajapati99/RuralChatbot/tree/main/chatbot-app

Backend (Node.js + Express):
https://github.com/nitishprajapati99/RuralChatbot/tree/main/Chatbot-Server

Database Models (MongoDB):
https://github.com/nitishprajapati99/RuralChatbot/blob/main/Chatbot-Server/DB/MongoDb.js

API End points:
https://github.com/nitishprajapati99/RuralChatbot/blob/main/Chatbot-Server/index.js

Setup Instructions (in README)
https://github.com/nitishprajapati99/RuralChatbot/blob/main/README.md

Table of Content

1. Introduction

India is a nation of extraordinary linguistic, cultural, and socioeconomic diversity. The country's rural population — estimated at over 900 million people across more than 640,000 villages — represents one of the world's largest communities with limited access to digital services. While the Digital India initiative has made significant strides in improving e- governance infrastructure over the past decade, the benefits of these improvements have largely accrued to urban, educated, and digitally literate citizens.

The central government has launched hundreds of welfare schemes targeting farmers, women, tribals, the economically weaker sections, the elderly, and rural youth. Schemes such as PM-KISAN, PM Awas Yojana, Pradhan Mantri Ujjwala Yojana, MGNREGS, and Ayushman Bharat represent massive financial commitments to rural welfare. Yet studies consistently show that awareness of and enrollment in these schemes remains far below potential — not because the schemes are in accessible, but because information about them is.

Most official government portals require a level of digital literacy, English or formal Hindi proficiency, and stable internet access that a significant proportion of rural citizens do not possess.

A farmer in rural Uttar Pradesh who speaks Bhojpuri, or an elderly woman in a remote Rajasthan village who has never used a smartphone, simply cannot navigate these portals — no matter how good the schemes behind them are.

Intermediaries — local agents, pradhan office staff, and informal consultants — have filled this information vacuum, but often at a cost. Misinformation, bribery, and selective access to benefits are well-documented consequences of this intermediary dependency.

The **Rural Chatbot** is designed to eliminate this information gap. It is an AI-powered, multilingual, voice-enabled web application that allows rural citizens to ask questions about government schemes in their own language — by typing or speaking — and receive clear, personalized, and accurate answers.

The system draws on modern artificial intelligence, natural language processing, and accessible web design to create a digital bridge between the government and India's rural population.

2. Problem Statement

Despite the Indian government's significant investment in welfare schemes for rural citizens, the effective reach of these programs is severely limited by an information delivery failure. The following specific problems define the challenge this project addresses:

1. Language and Literacy Barriers

Official government portals are predominantly available in English or formal Hindi. Rural India speaks over 19,500 dialects and hundreds of regional languages. A farmer speaking Bhojpuri, Awadhi, Bundeli, or Chhattisgarhi has no effective channel to access information in their native tongue. Low formal literacy rates further compound this problem.

2. Complex and Inaccessible Digital Portals

Existing government websites (NIC portals, state government sites) are text-heavy, have complex navigation menus, require multi-step form filling, and are not optimized for low-bandwidth mobile connections. Users with limited digital experience find them impossible to navigate independently.

3. Intermediary Dependency and Corruption.

In the absence of accessible information channels, rural citizens depend on local intermediaries — agents, panchayat staff, or unofficial consultants — who charge fees, provide inaccurate information, or selectively disclose scheme eligibility. This leads to corruption and exclusion of the most vulnerable beneficiaries.

4. Unclaimed Welfare Benefits

The CAG and NITI Aayog have documented thousands of crores in unspent welfare funds annually — directly attributable to low awareness and enrollment. This represents not just a financial waste but a fundamental failure of social justice.

5. Lack of Personalized Guidance

Even when citizens manage to access information, they receive generic scheme lists rather than personalized guidance. A 65-year-old tribal farmer from Jharkhand has very different eligibility from a 25-year-old woman entrepreneur in Bihar — yet both receive the same undifferentiated information.

6. No Voice-Based Information Access

Citizens with low literacy or poor eyesight have no voice-enabled channel through which they can access government scheme information in their language. Telephone helplines are often congested, staffed inconsistently, and limited to a few languages.

3. Proposed Solution

TheGraminChatbotproposesa comprehensive AI-powered solution to the information access problem faced by rural citizens. The system is designed around four core principles: **accessibility, accuracy, personalization, and scalability**.

3.1 System Overview

The Gramin Chatbot is a web-based multilingual conversational system. Users — whether on a smartphone, shared community computer, or public kiosk — can interact with the chatbot through text or voice. They can ask questions such as:

• "Kisan ke liye kya yojana hai?" (What schemes are available for farmers?)

• "PM Awas Yojana ke liye kaun eligible hai?" (Who is eligible for PM Awas Yojana?)

• "Mujhe ujjwala yojana ka cylinder kaise milega?" (How do I get a cylinder under Ujjwala Yojana?)

• "Old age pension ke liye apply kaise kare?" (How to apply for old age pension?)

The chatbot processes these queries using the Google Gemini NLP API, identifies the user's intent, matches their profile against the eligibility engine, and returns a clear, personalized response in the user's preferred language — as both text and spoken audio.

3.2 Core Modules

Module	Technology	Functions
Interface User	React.js, Bootstrap 5	Mobile-first chat UI, language selector, voice toggle
Backend	Node.js, Express.js	RESTful API, business logic,middleware
Database	MongoDB,Mongoose	Schemes, user profiles, sessions, FAQs
Engine Voice API NLP	Google Gemini API	Language understanding
Engine Voice I/O	Web Speech API	Response generation Speech-to-text input text-to-speech output
Eligibility Engine	Custom Service (Node.js).	Profile-based scheme filtering and ranking
Auth System , Admin Panel	JWT, bcrypt React.js (protected)	Secure user authentication and sessions,Content management for schemes and FAQs

3.3 How the System Works — End-to-End Flow

Step 1: The user opens the web application on their phone or a shared kiosk device.
Step 2: They select their preferred language from the language dropdown (Hindi, English, or regional).
Step 3: They type or speak their query into the chat interface.

Step 4: The Web Speech API converts voice to text (if using voice mode).

Step 5: The frontend sends the text query to the Express.js backend via secure API call with JWT authentication.

Step 6: The backend's Language Processing Module passes the query to the Google Gemini NLP API.

Step 7: Gemini identifies the intent and extracts key entities (scheme type, location, demographic).

Step 8: The Eligibility Engine retrieves the user's profile from MongoDB and matches it against scheme eligibility rules.

Step 9: Relevant schemes are retrieved from the MongoDB scheme database, ranked by relevance.

Step 10: The NLP model generates a conversational, easy-to-read response in the user's language.

Step 11: The response is returned to the frontend and displayed as a chat message.

Step 12: If voice mode is active, the Web Speech API reads the response aloud in the selected language.

4. Objectives of the Project

The Rural Chatbot project is guided by the following primary and secondary objectives, which together define the scope and success criteria of the system.

4.1 Primary Objectives

Multilingual Information Access: Build an AI-powered chatbot capable of understanding and responding in Hindi, English, and major Indian regional languages, ensuring that language is not a barrier to accessing government scheme information.

Voice-Enabled Interaction: Implement speech-to-text and text-to-speechfunctionality so that users with low literacy or visual impairments can interact fully with the system without needing to read or write.

Personalized Eligibility Recommendations: Develop a profile-based eligibility engine that filters and ranks government schemes based on individual user attributes, providing personalized guidance rather than generic scheme lists.

Direct Access Without Intermediaries: Eliminate the need for agents, middlemen, or pradhan office dependency by giving citizens direct, reliable, and accurate access to government scheme information.

Accessible Interface for Low-Literacy Users: Design a frontend that works for users with minimal digital literacy — large buttons, icon-driven navigation, local language labels, and a minimal text footprint.

4.2 Secondary Objectives

- Build a scalable, modular architecture that can easily integrate new languages, states, and government schemes.
- Implement an admin content management panel so that scheme data can be updated by non-technical staff.
- Ensure the system performs reliably on low-bandwidth 4G connections common in rural areas.
- Demonstrate the application of SOLID design principles and service-based architecture in a social-impact project.
- Provide a foundation for future integration with official government APIs and kiosk deployment.
- Support the national Digital India and Make in India missions through inclusive technology design.

5. Scope of the Project

5.1 In Scope

•Information and eligibility guidance for major central government welfare schemes across agriculture, housing, education, women welfare, health, and employment sectors.

• Multilingual query handling for Hindi, English, and major Indian regional languages viathe Gemini NLP API.

• Voice-to-text input and text-to-speech output for hands-free, eyes-free interaction.

• User profile system storing age, income, occupation, category, state, district, and gender for personalized recommendations.

• Session-based multi-turn conversation handling so follow-up questions are understood in context

• Admin panel for authorized users to add, update, and delete scheme records and FAQ entries.

• Responsive web interface optimized for mobile devices (Android smartphones) withlow-bandwidth support.

• JWT-based user authentication, rate limiting, and input validation for security.

5.2 Out of Scope (Current Version)

• DirectintegrationwithofficialgovernmentAPIs (e.g., PM-KISAN portal, DigiLocker) — planned for future release.

• Offline operation on devices without internet — addressed in the kiosk concept for future development.

• Native mobile app (iOS/Android) — the current version is a progressive web app accessible from any browser.

• State-level scheme databases beyond the covered central schemes — modular design allows future expansion.

• Automatic scheme application submission — the current system provides information and guidance only.

5.3 Target Users

User Group	Primary Use Case
Rural farmers	Inquire about agricultural subsidies, PM-KISAN, crop insurance
Rural women	Discover schemes for women entrepreneurs, maternal health

User Group	Primary Use Case
BPL households	Find food security, housing, and employment schemes (MGNREGS, PM
Senior citizens	Awas) Inquire about old age pension, Ayushman Bharat, senior welfare
Rural youth Panchayat	schemes Discover skill development, scholarship Schemes
workers	Use admin panel to help citizens and update local scheme information

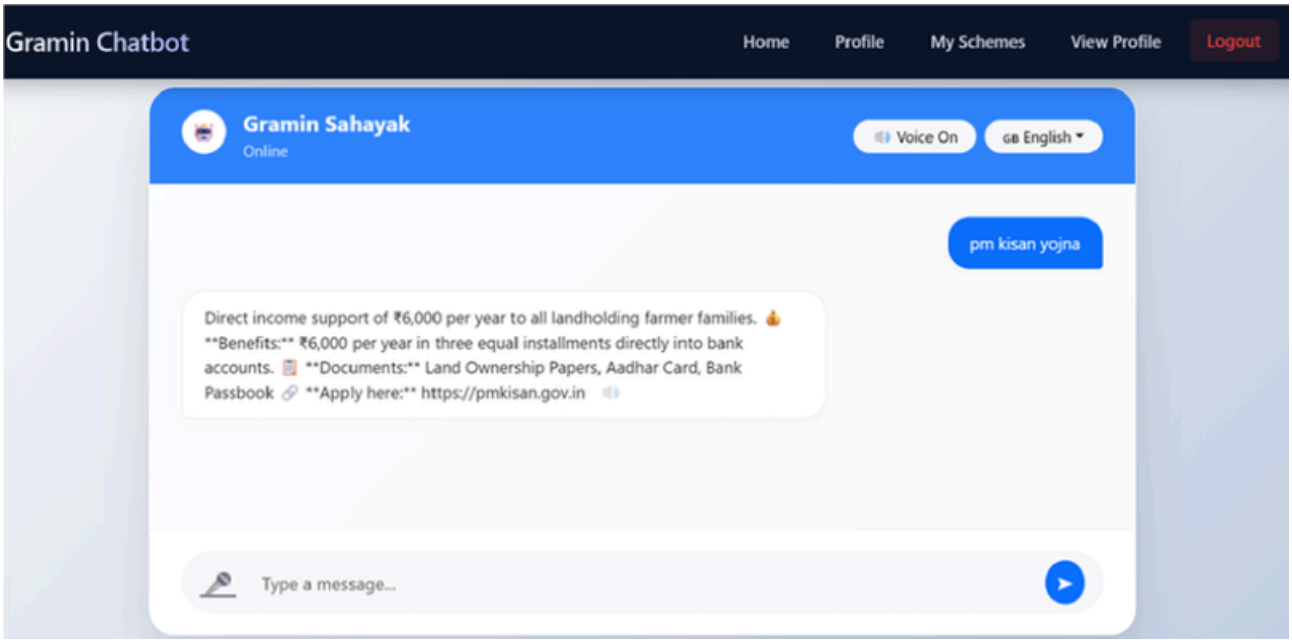


Figure (5.6.i)
Web page of Project

6. Why This Topic Was Chosen

The selection of the Rural Chatbot as our final year project was driven by a convergence of social awareness, technical interest, and national relevance. As engineering students from Ghaziabad — a city that sits at the boundary of urban infrastructure and rural Uttar Pradesh — we have direct personal exposure to the information divide that this project addresses.

6.1 Social Motivation

During our interactions with family members and community acquaintances from rural backgrounds, we repeatedly encountered the same story: eligible beneficiaries who had not received — and in many cases did not know about — government schemes that could meaningfully improve their lives. A farmer unaware of PM-KISAN, a woman unaware of Ujjwala, an elderly person unaware of old age pension — these are not hypothetical. They are the norm, not the exception.

We wanted to apply our technical skills to address this gap directly. AI and NLP technology has matured to the point where building a multilingual conversational system is within reach of an academic team — and the social impact of doing so could be significant.

6.2 Technical Motivation

The project offered an opportunity to apply a wide range of our curriculum learnings — machine learning, natural language processing, full-stack web development, database design, and API integration — in an integrated, real-world system. Working with the Google Gemini API gave us hands-on experience with state-of-the-art generative AI in a production context.

The engineering challenges — multilingual NLP, eligibility logic, voice I/O, performance on low-end devices, and scalable architecture — made this a technically rich and rewarding project.

6.3 NationalAlignment

The project directly supports three national priorities: the **Digital India** mission (expanding digital access and literacy), the **Make in India** initiative (developing indigenous technology for Indian social problems), and the **Jan Dhan Yojana** philosophy of inclusive access to government benefits. We believe technology built specifically for the Indian context — its languages, its connectivity constraints, its users — is essential for these missions to succeed at the last mile.

7. Literature Review

A review of existing research and systems in the domain of conversational AI for rural information access reveals both the potential and the gap that the Rural Chatbot seeks to fill.

7.1 Existing Chatbot Systems for Government Services

Several countries have deployed government chatbots for citizen service automation. Singapore's Ask Jamie, Australia's myGov virtual assistant, and Estonia's e-Residency chatbot are frequently cited examples. In India, the UMANG app and DigiLocker have virtual assistants, but these are limited to formal Hindi and English and assume a level of digital literacy that is not representative of rural India.

MyScheme (myscheme.gov.in), launched by the Government of India in 2022, represents the closest official effort to centralize scheme information. However, it operates as a search portal rather than a conversational system, requires formal language proficiency, and has no voice interface or regional language support beyond major scheduled languages.

7.2 MultilingualNLPforIndianLanguages

Research in Indian language NLP has accelerated significantly with the work of AI4Bharat (IIT Madras) on IndicNLP, Indic-BERT, and IndicBART models, which provide transformer- based language models for 12 major Indian languages. Google's Gemini and its predecessor models also incorporate substantial multilingual training data covering Indian languages.

Studies such as 'Multilingual NLP for Low-Resource Indian Languages' (ACL 2022) highlight that while transformer-based models perform well on high-resource Indian languages (Hindi, Bengali, Tamil), performance on dialectal variants and code-switched speech (mixing two languages) remains a research challenge — which the Rural Chatbot addresses through prompt engineering and fallback handling.

7.3 Voice Interfaces for Low-Literacy Users

Research by the MIT Media Lab's India team and IIT Bombay's HCI group has consistently demonstrated that voice-first interfaces dramatically improve usability for low- literacy populations in India. Projects such as Gram Vaani's community radio platform and Awaaz.De's IVR systems have shown that even users with no prior digital experience can effectively use voice-based information systems when properly designed.

The Ruraal Chatbot builds on these findings by integrating voice I/O through the Web Speech API, which provides cross-browser speech recognition and synthesis without requiring a separate IVR infrastructure.

7.4 Chatbot Architecture and Design Patterns

The Rural Chatbot draws on established patterns in chatbot architecture literature. The Rasa framework's approach to intent-entity-action architecture, Microsoft Bot Framework's dialog management patterns, and Google's Dialog flow multi-turn conversation handling informed the design of the chatbot's session management and fallback systems. The project adopts a simplified version of these patterns using the Gemini API's built-in context window for multi-turn handling.

7.5 Research Gap Addressed

While individual components — multilingual NLP, government chatbots, voice interfaces for rural users — have been studied independently, there is a notable lack of integrated, deployable systems that combine all three for the specific use case of Indian government welfare scheme information access. The Gramin Chatbot addresses this gap by building a production-ready, full-stack system that integrates all these components in a single accessible platform.

8. Feasibility Study

8.1 Technical Feasibility

Thep rojectusesreadily available, well-documented, open-source technologies. Node.js, React.js, and MongoDB are among the most widely used web development technologies globally, with extensive community support and documentation. The Google Gemini API provides a stable, well-documented interface for NLP tasks. All components are compatible with standard web hosting infrastructure and can be deployed on commodity cloud services.

The system is designed to run on low-end Android smartphones with 4G connectivity — adevice profile that is now widespread in rural India following the Jio revolution. The React.js frontend is built as a Progressive Web App (PWA), reducing data usage and improving performance on slow connections.

8.2 Economic Feasibility

Developmentcostis minimaldueto the use of free, open-source software. The only significant recurring cost is cloud hosting and the Gemini API. At the MVP stage, these costs are minimal:

Cost Item	Estimated Cost
Development Tools	Free (VS Code, GitHub, Postman — open source)
MongoDB Atlas	Free tier (512 MB) — sufficient for MVP
Render (Backend)	Free tier — upgradeable at ~$7/month
Vercel (Frontend)	Free tier — unlimited deployments
Gemini API	Free tier: 15 RPM, 1M tokens/day; paid tiers available

8.3 Operational Feasibility

The system can be operated and maintained by a small team. The admin panel allows non-technical staff to update scheme content without developer intervention. Panchayat workers, NGO staff, or government extension workers can be trained to use the admin panel in a short workshop. The chatbot itself requires no maintenance interaction from users — they simply open the web page and start chatting.

8.4 Social and Legal Feasibility

The project does not collect any personally identifiable information beyond a phone number or username for login, and a voluntary demographic profile. Data is stored securely and is not shared with third parties. The system provides information about government schemes but does not submit applications on users' behalf, keeping it outside the scope of regulated financial or legal services. Privacy policies comply with the IT Act, 2000 and relevant data protection guidelines.

8.5 DFD

This diagram shows how the user interacts with the system. The request is sent to the backend API, which processes the request and communicates with the MongoDB database to fetch or store data.

1) DFD LEVEL 0

The Data Flow Diagram (DFD) Level 0 represents the high-level overview of the Rural Chatbot system. It illustrates how the user interacts with the system and how data flows between the main components.

In this system, the user sends requests through the frontend interface, which are processed by the backend server. The backend communicates with the database to retrieve or store relevant information such as user data, schemes, and chatbot responses. The processed response is then sent back to the user.

This diagram helps in understanding the overall system flow without going into implementation details.

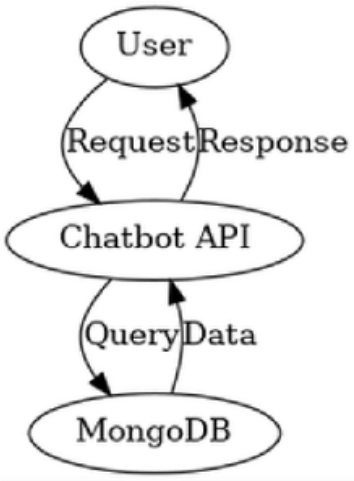
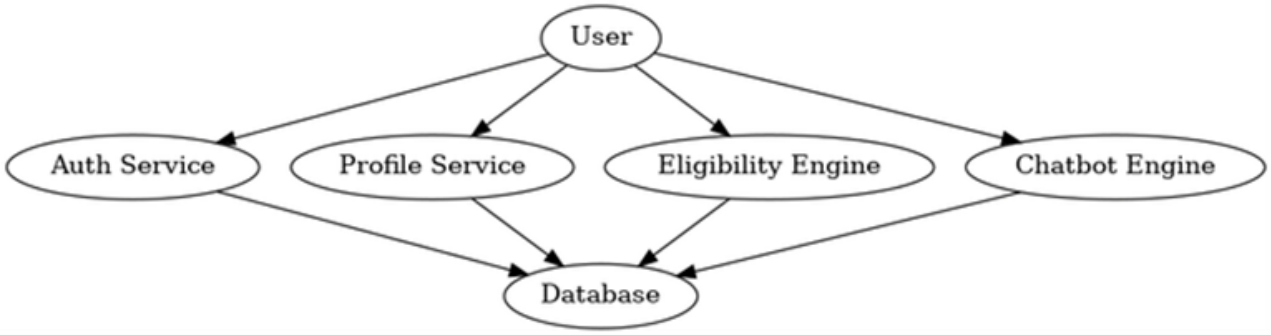


Figure (8.5.i)

2) DFD LEVEL 1

The DFD Level 1 provides a more detailed view of the internal working of the Rural Chatbot system. It breaks down the main system into smaller functional components such as authentication service, profile management, chatbot processing, and eligibility engine.

- Each component performs a specific task:
- The authentication service manages user login and registration.
- The profile service handles user data updates.
- The chatbot engine processes user queries and retrieves answers.
- The eligibility engine filters schemes based on user profile data.
- This level of diagram helps in understanding how different modules interact with the database and with each other to deliver accurate and personalized results.



Figure(8.5.ii)

9. System Architecture

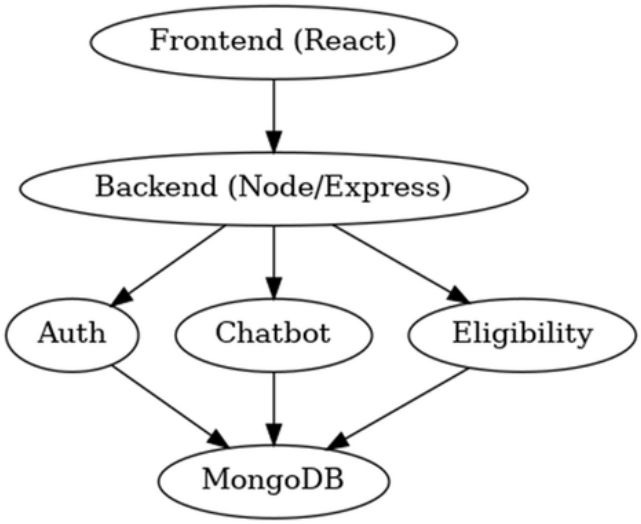
System Architecture Diagram

The System Architecture Diagram illustrates the overall structure and components of the Rural Chatbot system. It shows how the frontend, backend, and database interact with each other.

The frontend is developed using React.js and provides the user interface. The backend, built using Node.js and Express.js, handles business logic and API requests. The backend is further divided into different services such as authentication, chatbot processing, and eligibility filtering.

All data is stored and managed in MongoDB, which ensures scalability and efficient data handling. This architecture follows a modular approach, making the system easy to maintain, scalable, and efficient.

This diagram provides a clear understanding of how different layers of the system are connected and how data flows across them..



Figure(9.i)

9.3 API Communication Pattern

All frontend-to-backend communication follows a standard RESTful pattern:

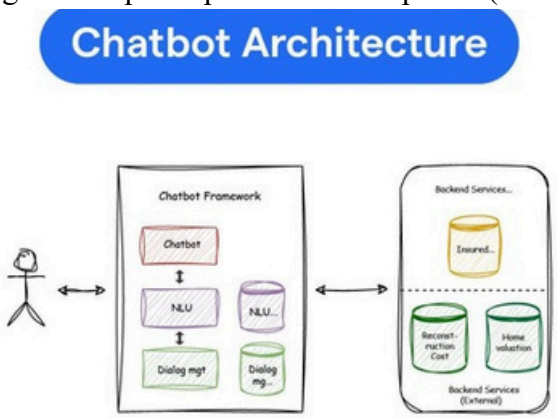
• Base URL: https://gramin-chatbot-api.onrender.com/api/v1/

• Authentication: Bearer JWT token in Authorization header

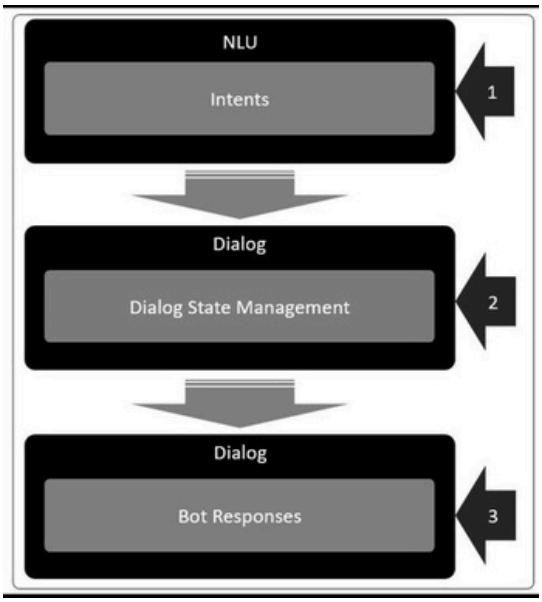
• Content-Type: application/json

• Error handling: Standardized error response objects with code and message fields

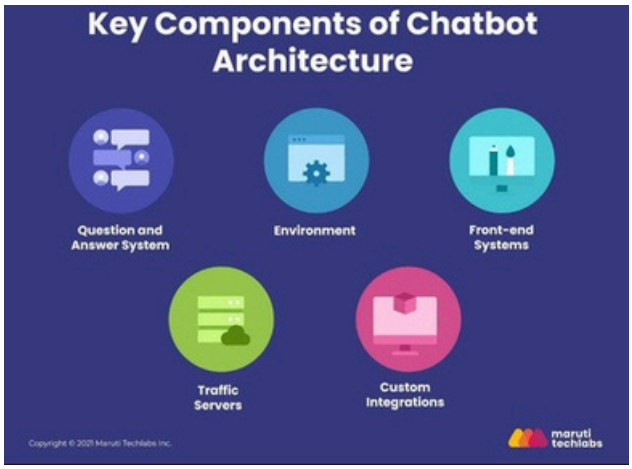
• Rate limiting: 100 requests per 15 minutes per IP (configurable)



Fgure (9.3.i)



Fgure (9.3.ii)



Fgure (9.3.iii)

10. Backend Architecture

The backend is the core processing layer of the Rural Chatbot. Built on Node.js with the Express.js framework, it implements a service-based architecture that strictly adheres to SOLID principles. Each functional domain is encapsulated in its own service module, enabling independent development, testing, and scaling.

10.1 Directory Structure

```
■■■ Rural-Server/
    src/
■■■■
      config/ # DB connection, environment config controllers/ #
■■ ■
      Route handler functions middleware/ # Auth, rate limiter,
■■ ■
      validator, error handler models/ # Mongoose schemas (User,
■■ ■
      Scheme, Chat, etc.) routes/ # Express route definitions
■■■■
      services/ # Business logic services
■■■■
        authService.js
■ ■ ■■■
        chatService.js
■ ■ ■■■
        eligibilityService.js
■ ■ ■■■
        schemeService.js
■ ■ ■■■
        nlpService.js
■ ■ ■■■
      utils/ # Helper functions, constants
■■ ■■
    tests/ # Unit and integration tests
■■■
    server.js # Application entry point
■■■
```

10.2 Service Modules

Service	Responsibility
authService.js	Handles user registration, login, JWT generation, token refresh, and logout.
chatService.js	Receives user queries, manages conversation context, coordinates NLP calls,
eligibilityService.js	Interfaces with the
schemeService.js	Loads user profile, evaluates scheme eligibility rules matching CRUD operations
userProfileService.j	for the scheme database: create, read, update, delete, search
s adminService.js	Manages user demographic profile storage and retrieval.
	Provides admin-level operations: bulk scheme import, FAQ management, usage

10.3 Middleware Stack

Everyrequestpassesthroughthe following middleware chain:

CORS Middleware: Allows requests only from trusted frontend origins.

Body Parser: Parses JSON request bodies.

Rate Limiter: Enforces per-IP request limits to prevent abuse.

Auth Middleware: Validates JWT token on protected routes; rejects unauthorized requests.

Input Validator: Validates and sanitizes request body fields; prevents injection attacks.

Error Handler: Catches all thrown errors and returns standardized error responses.

Request Logger: Logs method, path, status code, and response time for monitoring.

11. Frontend Architecture

The frontend is a React.js single-page application (SPA) designed with rural usability as its primary constraint. Every design decision — from font size to button placement to language of labels — is made with the target user in mind: a first-time smartphone user in a village who may have limited literacy and is accessing the chatbot through a 4G connection.

11.1 Design Principles

Mobile-First: All layouts are designed for 360–414px mobile screens first and scale up to desktop.

Minimal Cognitive Load: The interface presents one task at a time. The chat screen shows only the message input and conversation — nothing else.

Large Touch Targets: All buttons are at minimum 48x48px, meeting accessibility guidelines for users with poor motor control.

High Contrast: Text-to-background contrast ratios meet WCAG AA standards for users in bright outdoor sunlight.

Local Language Labels: Navigation labels and instructions appear in the user's selected language, not just English.

Offline Awareness: The app detects when the connection is lost and shows a clear,friendly error rather than a technical message.

11.2 Component Structure

Component	Description
App.jsx	Root component; manages routing and global state (language, auth status)
ChatScreen.jsx	Main chat interface: message list, input field, voice button, language selector
MessageBubble.jsx	Renders individual chat messages with timestamp and sender indicator
VoiceButton.jsx	Toggles speech recognition; shows listening animation when active
LanguageSelector.jsx	Dropdown to choose preferred language; stores selection in localStorage
SchemeBrowser.jsx	Searchable, filterable grid of government schemes by category

Component	Description
ProfileForm.jsx	Form for entering demographic profile data used by eligibility engine
AdminDashboard.jsx	Protected admin interface for managing schemes and FAQs
LoginPage.jsx	Simple login form with phone number and OTP or password authentication
SchemeCard.jsx	Individual scheme card: name, one-line summary, eligibility tags

11.3 State Management

Global state is managed using React Context API with use Reducer, avoiding the complexity of Redux for an application of this scale. Key global state includes: current user (auth token, profile), selected language, and conversation history. Component-level state is used for local UI concerns such as form field values and loading indicators.

12. Database Design

MongoDB is chosen as the primary database for its document model flexibility, which is well-suited to storing heterogeneous government scheme data where different schemes have different fields, rules, and structures. Mongoose ODM provides schema validation, virtuals, and query building.

12.1 Collections

Collection	Purpose	Approx. Documents
users	Stores user login credentials and role	Thousands
user_profiles	Demographic data for the eligibility engine	Thousands
schemes	Government scheme master data Conversation	2,000
chat_sessions	history per user session Individual chat	500– Millions
messages faqs	messages linked to sessions Pre-answered common questions for fast retrieva2l00–500	(archived) Millions
admins	Admin user accounts with role-based permissionsDozens	

12.2 Key Schema Designs

Scheme Document

– _id:O bjectId (auto-generated primary key)

– name: String (scheme name in English)

– name_hindi: String (scheme name in Hindi)

– description: String (detailed description)

– short_description: String (one-line summary for cards)

– category: String (enum: agriculture | housing | education | health | women | employment)

– tags: [String] (searchable keywords, e.g., ['farmer', 'kisan', 'subsidy'])

– eligibility_rules: Object { min_age, max_age, max_income, gender, categories[], states[] }

– benefits: String (what the beneficiary receives)

– application_process: String (how to apply)

– documents_required: [String]

– ministry: String

– launch_date: Date

– is_active: Boolean

– created_at, updated_at:

Date

User Profile Document

– userId: ObjectId (ref: users)

– age: Number

– annual_income: Number (in INR)

– occupation: String (enum: farmer | daily_wage | artisan | student | homemaker | other)

– category: String (enum: SC | ST | OBC | GEN | EWS)

– gender: String (enum: male | female | other)

– state: String

– district: String

– is_bpl: Boolean

– updated_at: Date

12.3 Indexing Strategy

Collection	Indexed Fields	Reason
schemes	category, tags, is_active	Fast scheme retrieval by category and tag filters
user_profiles	userId	Fast profile lookup by user ID
chat_sessions	userId, created_at	Fast session retrieval; time-based archiving
faqs	tags, category	Fast FAQ matching for common queries

12.4 User Database

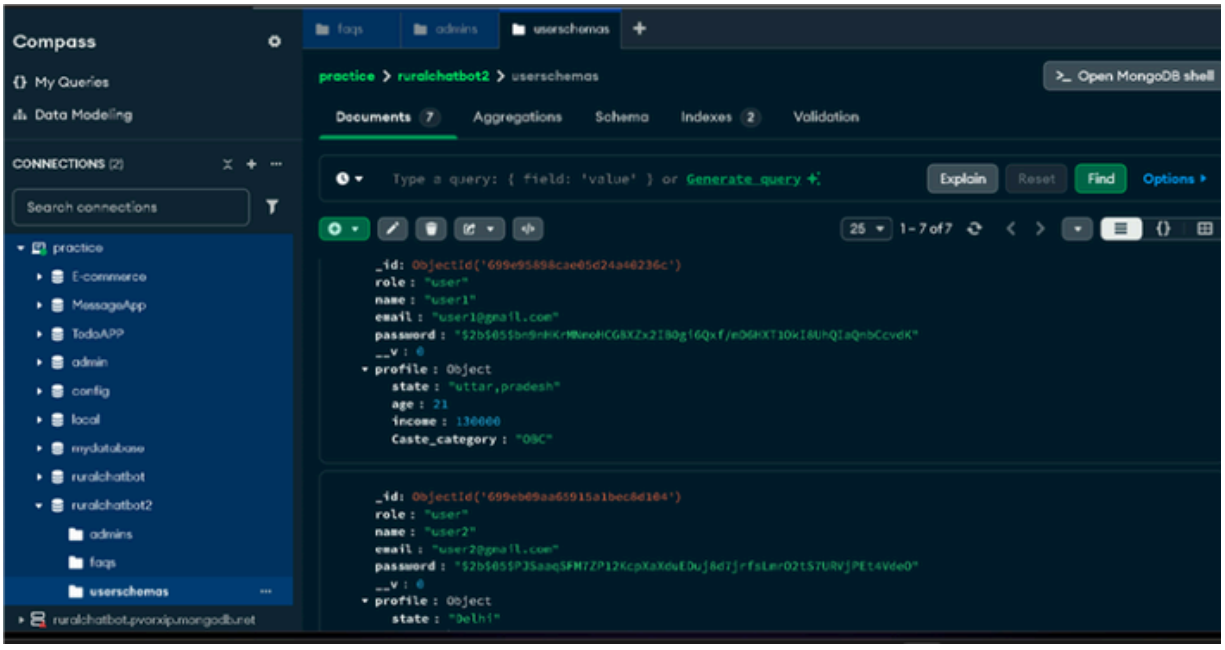
The User schema is designed to store both authentication details and profile information required for personalized services in the Rural Chatbot system.

The schema includes basic authentication fields such as name, email, and password, which are used for user registration and login. A role field is also included to differentiate between normal users and administrators.

In addition to authentication, the schema contains a profile object that stores important user attributes such as age, income, state, category, gender, and occupation. These fields are essential for the eligibility engine, which filters government schemes based on user-specific criteria.

An isProfileComplete flag is used to ensure that users provide all required information before accessing personalized features like scheme recommendations.

This integrated schema design improves performance, simplifies queries, and enables efficient filtering of schemes based on user data.



Fgure (12.4.i)

12.5 FAQ Schema

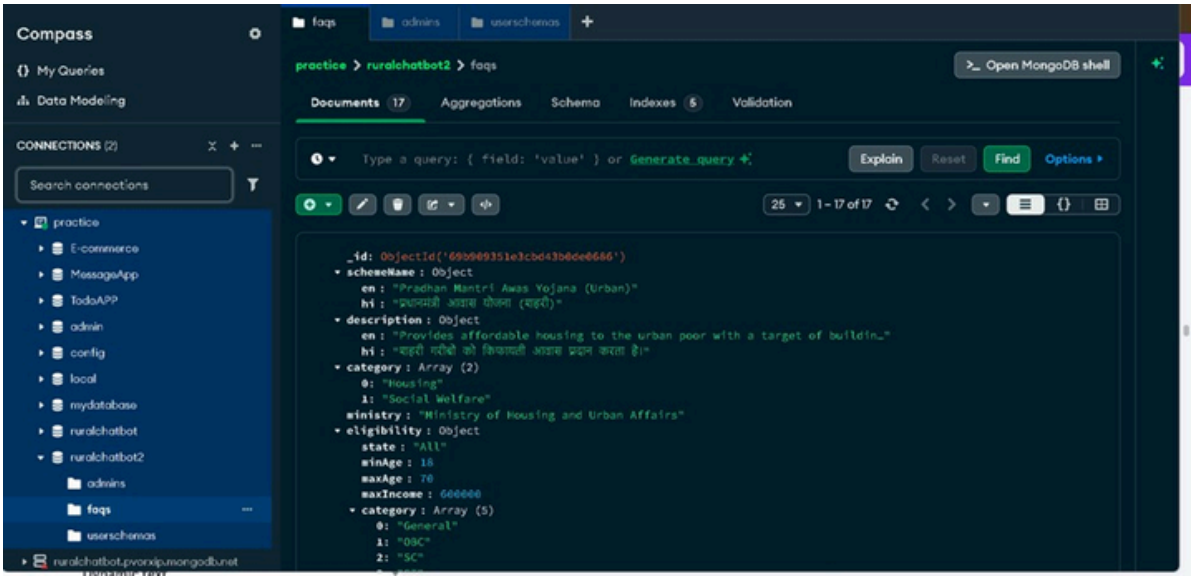
The FAQ schema is designed to store frequently asked questions and their corresponding answers for the chatbot system. It plays a crucial role in enabling the chatbot to provide quick and accurate responses to user queries.

Each FAQ entry consists of a question and its answers in multiple languages, including English and Hindi. This ensures accessibility for users from different linguistic backgrounds, especially in rural areas.

The schema also includes tags and synonyms to improve the search and matching process. Tags help categorize the FAQs, while synonyms allow the chatbot to understand different variations of the same question using fuzzy search techniques.

Additionally, a usageCount field is included to track how often a particular FAQ is used, which can help in optimizing and improving the chatbot's performance over time.

This structured design ensures efficient data retrieval, better user experience, and scalability of the chatbot system.



Fgure (12.5.i)

13. API Design

The backend exposes a versioned RESTful API. All endpoints are prefixed with /api/v1/. Protected endpoints require a valid JWT Bearer token in the Authorization header. All responses follow a standard envelope format with status, data, and message fields.

13.1 API Endpoints

Endpoint	Method	Auth		Description
/auth/register	POST	No		Register new user (phone + password)
/auth/login	POST	No		Login; returns JWT access token
/auth/refresh	POST	No		Refresh expired access token
/profile /profile	GET	Yes		Get current user's profile
/chat	PUT	Yes		Update user profile (demographics)
/chat/history	POST	Yes		Submit query; get AI-generated response
/schemes	GET	Yes		Get paginated chat history
/schemes/:id	GET	No		List schemes (filter by category/tag)
/eligibility /faqs	GET	No		Get a specific scheme
/admin/schemes	POST	Yes		Get personalized scheme recommendations
/admin/faqs	GET	No		List FAQs (filter by category)
/admin/stats				POST/PUT/DELETAEdminCreate/update/delete scheme records POST/PUT/DELETAEdminCreate/update/delete FAQ entries
	GET			AdminUsage statistics and analytics

13.2 Standard Response Envelope

```
{
"status": "success" | "error",
"data": { ... },
"message": "Human-readable message",
"pagination": { "page": 1, "total": 50 } // for list endpoints }
```

14. Authentication & Session Management

The system uses JSON Web Token(JWT) based stateless authentication. This approach is well suited to a mobile-first application where maintaining server-side sessions would add unnecessary state management complexity.

14.1 Authentication Flow

• User submits phone number and password to POST /api/v1/auth/login.

• The server validates credentials and, if correct, generates a signed JWT (access token, 1-hour expiry) and a refresh token (7-day expiry).

• The JWT is returned to the client and stored in memory (not localStorage, to prevent XSS token theft).

• Every subsequent protected API call includes the JWT in the Authorization: Bearer header.

• The auth middleware validates the JWT signature and expiry on every protected request.

• When the access token expires, the client uses the refresh token to obtain a new access token without requiring re-login.

14.2 Password Security

• Passwords are hashed using bcrypt with a salt factor of 10 before storage — plain-text passwords are never stored.

• The login endpoint is rate-limited to 5 attempts per 15 minutes per IP to prevent
brute-force attacks.

• Password reset is handled via OTP sent to the registered phone number.

15. User Profile System

The User Profile Sy stem is the data foundation for the Eligibility Engine. When a user first logs in, they are guided through a simple, icon-driven profile setup form. The profile collects the minimum demographic information needed to power personalized recommendations — nothing more.

15.1 Profile Attributes and Their Role

Attribute	Input Type	Used For
Age	Number slider (0-100)	Age-restricted schemes (senior, youth, child)
Annual Income	Dropdown (income ranges B)PL/.	APL/EWS category; income-capped schemes
Occupation	Icon-based selector	Farmer, artisan, student, daily-wage schemes
Category	Radio (SC/ST/OBC/GEN/ER) WesServation-based and caste-specific schemes	
Gender	Radio (Male/Female/OtherW)omen-only schemes; gender-neutral filtering	
State	Dropdown (Indian states)State-specific and region-targeted schemes	
District	Dropdown (filtered by stateD)istrict-level scheme targeting	
BPL Status	BYePsL/-Nspo etocigfigcl escheme eligibility	

Profiles are voluntary — users can use the chatbot without creating a profile, but they receive generic (non-personalized) scheme lists. The profile can be updated at any time. Users who do not want to create an account can use the chatbot in anonymous mode with a session-based temporary profile.

16. Eligibility Engine

The Eligibility Engine is the core intelligent component that distinguishes the Rural Chatbot from a simple FAQ or search system. It ensures that every user receives scheme recommendations that are relevant to their specific demographic and socioeconomic situation.

16.1 Engine Logic

The engine executes the following steps for each query:

1. Load the user's profile from MongoDB (or use the anonymous session profile if no login).

2. Extract the query intent and scheme category from the NLP layer (e.g., intent=scheme_inquiry, category=agriculture).

3. Query the schemes collection for all active schemes in the relevant category.

4. For each retrieved scheme, evaluate its eligibility_rules object against the user's profile attributes.

5. Compute a relevance score for each matching scheme based on how many eligibility criteria match.

6. Sort the matching schemes by relevance score (descending) and return the top 3–

7. Pass the ranked scheme list to the NLP layer for conversational response generation.

16.2 Eligibility Rule Evaluation

Each scheme's eligibility_rules object has the following structure and evaluation logic:

```
eligibility_rules: { min_age: 18, // user.age >= min_age max_age: 60, // user.age <= max_age max_annual_income: 200000,// user.annual_income <= max_annual_income gender: 'female', // user.gender === gender (null = any) categories: ['SC','ST'], // user.category in categories (null = any) states: ['UP','MP'], // user.state in states (null = all India) is_bpl: true, // user.is_bpl === is_bpl (null = both) occupation: ['farmer'], // user.occupation in occupation (null = any) }
```

16.3 Personalization Impact

The personalization impact of the eligibility engine is significant. Without it, a query about 'agricultural schemes' might return 50+ schemes. With the engine, a 55-year-old SC- category farmer from UP with income below Rs. 1.5 lakh receives a curated list of 5–7 schemes specifically relevant to their situation — PM-KISAN, PM Fasal Bima Yojana, Kisan Credit Card, and state-specific UP farmer schemes. This dramatically increases the actionability of the information provided.

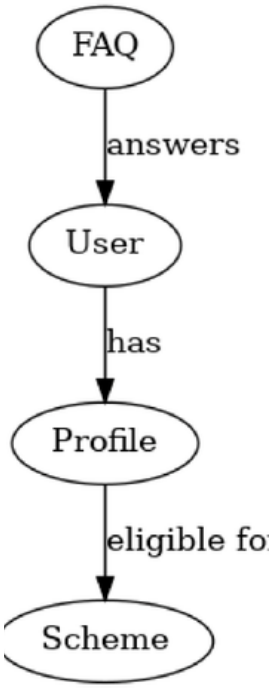
16.4 E-R Diagram

Entity Relationship Diagram (ER Diagram)

The Entity Relationship (ER) Diagram represents the structure of the database used in the Rural Chatbot system. It defines the entities, their attributes, and the relationships between them.

The main entities in the system include User, Profile, Scheme, and FAQ. Each entity stores specific information required for the functioning of the system. For example, the User entity stores authentication details, while the Profile entity contains personal data used for eligibility checking.

The relationships between these entities enable the system to provide personalized scheme recommendations and accurate chatbot responses. This diagram is essential for understanding how data is organized and managed within the system.



Figure(16.4.i)

17. Chatbot System & NLP Integration

The Chatbot System is the conversation all layer that ties together the NLP API, eligibility engine, and the scheme database into a coherent, user-friendly the experience.

17.1 Conversation Flow Architecture

The chatbot maintains conversation context using the Gemini API's built-in context window. Each API call to Gemini includes the last N messages of the conversation, allowing the model to understand follow-up questions in context.

17.2 System Prompt Engineering

The chatbot's behavior is defined through a carefully crafted system prompt sent to the Gemini API with every request. The system prompt instructs the model to:

• Always respond in the language of the user's query (Hindi, English, or regional dialect).
• Keep responses simple, conversational, and jargon-free — suitable for users with low formal education.
• When a user asks about a scheme, ask one clarifying question to understand their situation before recommending.
• Always mention: (a) what the scheme provides, (b) who is eligible, and (c) how to apply.
• If unsure about a scheme's details, acknowledge uncertainty and suggest the user visit the nearest Jan Seva Kendra (Common Service Centre).
• Never provide legal, financial, or medical advice beyond government scheme information.
• When the user's query is outside the domain of government schemes, politely redirect.

17.3 Intent Categories

Scheme	**Example Query**	**System Action**
Inquiry	Kisan ke liye kya scheme hai?	Eligibility engine + scheme list
Eligibility Check	Kya main PM Awas ke liye eligible Per ohfoiloen?	match against specific scheme
Application Guidance	PM KISAN mein apply kaise kare?	Retrieve application_process field
Benefit Query	MGNREGS mein kitna milta hai?	Retrieve benefits field

Intent	Example Query	System Action
Document Query	Ayushman card ke liye kya chahiye?	Retrieve documents_required
General Help	field Chatbot ka use kaise karte hain?	Help/onboarding response
Fallback	Unrecognized query	Graceful fallback + CSC suggestion

17.4 Fallback and Error Handling

•If the Gemini API returns an error or times out, the chatbot returns a friendly message and retries once.

• If the intent cannot be determined with sufficient confidence, the chatbot asks a clarifying question.

• If no schemes match the user's eligibility profile, the chatbot explains why and suggests visiting a CSC.

• A 'Did you mean?' suggestion is offered if the query contains a recognized scheme
name with a minor spelling variation.

18. Multilingual Support

Multilingual support isfoundational to the Gramin Chatbot's mission. India's linguistic diversity is one of the world's richest — with 22 scheduled languages and thousands of dialects — and it is precisely this diversity that makes rural information access so challenging with conventional systems.

18.1 Language Coverage

Language	Script		Support LevelNotes
Hindi English	Devanagari	Full	Primary language; full NLP + voice + UI Fallback
Hinglish	Latin Mixed	Full	language for urban/educated users Code-switching
Bengali	Bengali	Full	between Hindi and English NLP via Gemini; UI
Marathi	Devanagari	Partial	labels in progress NLP via Gemini; voice support
Tamil	Tamil	Partial	via browser NLP via Gemini; voice support via
Telugu	Telugu	Partial	browser NLP via Gemini; voice support via
Gujarati	Gujarati	Partial	browser NLP via Gemini; planned for next release
Bhojpuri	Devanagari	Partial	ExperimentalDialectal variant of Hindi; Gemini handles well
Awadhi/Bundel	Devanagari		ExperimentalRegional dialects; tested with common vocabulary

18.2 VoiceInput and Output

Voice interaction is implemented using the browser's native Web Speech API, which provides both SpeechRecognition (voice-to-text) and SpeechSynthesis (text-to-speech). This approach avoids the need for a separate voice service infrastructure. The language for recognition and synthesis is set dynamically based on the user's selected language preference.

• **SpeechRecognition:** Activated by pressing and holding the microphone button. Supports continuous speech with interim results shown in real-time. Works offline for short phrases on Chrome Android.

• **SpeechSynthesis:** Reads chatbot responses aloud in the selected language. Speed and pitch are adjustable. Stops automatically when the user starts typing or presses a stop button.

19. Admin Panel

The AdminP anelisaprotected web interface accessible only to users with the 'admin' role. It enables government officials, NGO coordinators, or system administrators to keep the chatbot's knowledge base current without requiring developer involvement.

19.1 Admin Capabilities

Scheme Management: Add new schemes with all fields (name, description, eligibility rules, benefits, application process, documents required). Edit existing schemes when policy details change. Soft-delete outdated schemes (marked inactive but retained in DB).

FAQ Management: Add common questions and pre-written answers that the chatbot retrieves directly without making a Gemini API call (faster response, lower API cost). Organize FAQs by category and tag.

Usage Analytics: View dashboard with key metrics: total queries per day/week/month, most queried schemes, user language distribution, geographic heatmap of queries, and fallback rate.

User Management: View registered users, their profile completeness, and usage patterns. Promote users to admin role. Deactivate abusive accounts.

Bulk Import: Upload scheme data in bulk via CSV or JSON file — reducing data entry time when adding a new category of schemes.

19.2 Admin Security

• Admin routes require a role: 'admin' claim in the JWT — separate from regular user tokens.

• Admin login attempts are logged with IP address and timestamp.

• All admin actions (create/update/delete) are recorded in an audit log for accountability.

• Admin panel is hosted on a separate subdomain and is not linked from the public frontend.

20. Rate Limiting & Performance Optimization

20.1 Rate Limiting

Ratelimitingis implemented using the express-rate-limit package. Limits are configured per route type to balance usability against abuse prevention:

Route Group	Limit	Window	Purpose
POST /auth/login	5 requests	15 min	Prevent brute-force login attacks
POST /auth/Signup	3 requests	1 hour	Prevent account farming
POST /chat	60 requests	15 min	Generous limit for active chatting High
GET /schemes Admin	200 requests	15 min	limit for browsing
routes Global fallback	100 requests	15 min	Moderate limit for admin operations
	300 requests	15 min	Catch-all for unlisted routes

20.2 Database Performance

•MongoDBcompound indexeson(category, is_active) reduce scheme query time by ~85% vs. full collection scans.

• Mongoose's .lean() method is used on all read-heavy routes, returning plain JS objects instead of full Mongoose documents (40–60% faster).

• Database connection pooling (default Mongoose pool size: 5) prevents connection overhead on high-traffic deployments.

• Scheme data that changes infrequently (e.g., scheme master list) is cached in memory for 5 minutes using a simple in-memory LRU cache.

20.3 Frontend Performance

•React lazyloading andcodesplitting reduce the initial JavaScript bundle by ~60%, critical for slow 4G connections.

• Images (scheme category icons) are served in WebP format and lazy-loaded to reduce data usage.

• The Scheme Browser uses virtual scrolling (react-window) to render only visible items, supporting large scheme lists without performance degradation.

• Service Worker (PWA) caches static assets on first load, enabling faster subsequent visits and basic offline access to cached pages.

21. Scalability

While the Rural Chatbot is currently deployed as a single-server application, its architecture is designed from the ground up to scale — first vertically (more powerful server), then horizontally (multiple server instances), and ultimately to enterprise-grade infrastructure.

21.1 Horizontal Scalability

• The stateless JWT authentication means any backend instance can handle any request — no sticky sessions required.
• The service-based architecture allows individual services (e.g., the NLP service) to be extracted into separate microservices and scaled independently.
• MongoDB Atlas supports horizontal scaling via automatic sharding for datasets exceeding single-node capacity.
• Deployment on container orchestration platforms (Kubernetes) can automatically scale the number of backend pods based on traffic.

21.2 Content Scalability

• The scheme database schema is generic enough to hold schemes from any Indian state, ministry, or category without schema changes.
• New languages can be added by updating the Gemini prompt and adding language
options to the frontend selector — no backend changes required.
• The eligibility rule structure supports complex, nested conditions (age + income + category + state) without code changes — just data updates in the admin panel.

22. Security

Security is a critical aspect of any web-based application. The Rural Chatbot system handles sensitive user data such as login credentials, personal profile information, and interaction history. Therefore, multiple layers of security mechanisms have been implemented to ensure data protection, authentication, and safe communication between client and server.

22.1 Authentication Mechanism

- The system uses JSON Web Token (JWT) for authentication.
- Working: When a user logs in, the server generates a JWT token.
- The token contains: User ID Role (Admin/User) The token is sent to the client and stored in local Storage.
- For every protected API request, the token is sent in the request header:
- Authorization: Bearer <token>
- Benefit: Stateless authentication No need to store session data on server Fast and scalable

22.2 Authorization (Role-Based Access Control)

- The system implements role-based access control (RBAC).Roles:
- User → Can ask queries, view responses
- Admin → Can add FAQs, update chatbot knowledge
- Implementation: Middleware verifies JWT token
- Checks user role
- Grants or denies access
- Example:Only admin can access: POST /faq/add

22.3 Password Security

- User passwords are never stored in plain text.
- Implementation: Passwords are hashed using bcrypt
- During login: Entered password is compared with hashed password
- Benefits: Protects user data in case of database breach
- Prevents password theft

22.4 Data Validation & Sanitization

- To prevent malicious inputs:
- Frontend:Required fields validation
- Proper input formats (email, password)
- Backend:Request body validation
- Avoids invalid or harmful data
- Protection against:Invalid data insertion
- Application crashes

22.5 Secure API Communication

- All API requests include authentication headers Sensitive routes are protected using middleware
- Unauthorized access returns error responses

22.6 Token Management

- Tokens are stored in browser localStorage
- Token expiry can be implemented for better security
- Invalid or expired tokens deny access

22.7 Deployment Security

- The application is deployed on Render.com, which provides:
- Secure hosting environment
- HTTPS support
- Environment variable protection

22.8 Future Security Enhancements

- Implement HTTP-only cookies for token storage
- Add Two-Factor Authentication (2FA)
- Implement rate limiting to prevent brute force attacks
- Use helmet.js for HTTP security headers
- Encrypt sensitive user data

23. SOLID Principles in the Codebase

The Rural Chatbot backend strictlyadheres tothefive SOLID principles of object-oriented and service-oriented design, ensuring the codebase is maintainable, extensible, and testable.

S — Single Responsibility Principle

Each service module has exactly one domain responsibility. chatService.js handles NLP handles only Gemini API communication. No service crosses domain boundaries.

O — Open/Closed Principle

The eligibility engine is open for extension (new eligibility rule types can be added to the schema) but closed for modification (existing rule evaluation logic does not need to change). New languages are added by configuration, not code change.

L — Liskov Substitution Principle

The NLP layer is abstracted behind an interface. The Gemini API implementation can be replaced with any other NLP provider (OpenAI, Cohere, IndicNLP) without changing the chatService.js that calls it.

I — Interface Segregation Principle

Controllers expose only the methods needed by their routes. The admin controller exposes admin-only methods; the user controller exposes user-only methods. No controller is forced to implement methods irrelevant to its domain.

D — Dependency Inversion Principle

High-level modules (controllers) depend on abstractions (service interfaces), not on concrete implementations. This is achieved through Node.js module dependency injection, making unit testing straightforward.

24. UI/UX Design Principles

The Rural Chatbot's user interface is the product's most critical differentiator. Unlike most government digital services — which are designed for literate, English-proficient, urban users — this interface is explicitly designed for rural, low-literacy, first-time smartphone users.

24.1 Design Language

Design Element	Choice	Reasoning
Primary Color	Deep Blue (#1a3a5c)	Trust, government association; high contrast on white
Accent Color Font	Bright Blue (#2e75b6)	Highlights interactive elements; accessible contrast ratio Fast
Font Size	System default sans-serif	loading; familiar to device users
Button Height	16px base, 18px for key text	Readable in bright sunlight on small screens
Iconography Chat	Minimum 48px	WCAG touch target guideline; fat-finger friendly
Bubbles Error	Icon + label always pairedIcons alone are ambiguous for low-literacy users	
Messages Loading	User: right/blue; Bot: left/whSitetandard messenger convention; universally understood Plain	
State	language in selected laNnog tueacghenical jargon; tells user what to do next Animated dots (typing	
	indicaStoert)s expectation; reduces perceived wait time	

24.2 Onboarding Flow

New users are guided through a three-step onboarding process on first launch:

- **Language Selection:** Full-screen language picker with native script labels and audio pronunciation of each language name.
- **Quick Tutorial:** Three illustrated slides showing how to type a question, use the voice button, and browse schemes. Skippable.
- **Optional Profile Setup:** Friendly prompt to enter basic demographics to get personalized results. Users can skip and complete later.

24.3 Accessibility

- All interactive elements have ARIA labels for screen reader compatibility.
- Color is never the sole indicator of state — icons and text labels accompany color changes.
- Text-to-speech output makes the chatbot fully usable by visually impaired users.
- The interface functions correctly in grayscale mode for users with color vision deficiency.

25. Kiosk Concept — Future Feasibility Enhancement

Despite the rapid expansion of smart phone penetration in rural
India, a significant proportion of the target population — particularly the elderly and women in conservative households — does not own or have access to a personal smartphone. The Rural Chatbot Kiosk concept addresses this final mile of digital exclusion.

25.1 Kiosk Design Concept

Rural Chatbot Kiosks are stand alone touch screen stations designed for installation in high-footfall public locations in rural areas: village panchayat offices, ration shops (PDS outlets), Common Service Centres (CSCs), health sub-centres, and government bank branches.

25.2 Kiosk Hardware Specification (Proposed)

Component	Specification
Screen	15.6" capacitive touchscreen, 1080p, outdoor-readable brightness
Processor	Raspberry Pi 5 or equivalent ARM SBC
Storage	64 GB eMMC (offline scheme database cache)
Connectivity	4G SIM + Wi-Fi; falls back to local cache when offline
Audio Printer	Directional microphone + speaker (for voice interaction)
Enclosure	Optional: thermal printer for scheme info printout
Interface	Vandal-resistant metal enclosure with solar charging option Simplified
Future Scope	kiosk version of the web app (larger buttons, shorter text)

25.3 KioskSoftwareArchitecture

• The kiosk runs a locked-down Chromium browser in kiosk mode, displaying the Gramin Chatbot PWA.
• A local Node.js server maintains an offline-capable cache of the scheme database, synced daily with the central server.
• When internet is available, queries are processed by the central Gemini API. When offline, the local cache provides scheme information via keyword matching.
• Usage data is queued locally and synced to the analytics dashboard when connectivity is restored.
• Remote management: kiosk software, scheme content, and OS updates are deployed remotely via a secure management console.

26. Tools & Technologies Used

26.1 Technology Stack Summary

The Rural Chatbot system is developed using a modern full-stack web technology approach, ensuring scalability, performance, and ease of maintenance. The technology stack is divided into three main layers: frontend, backend, and database, along with deployment and development tools.

26.2 Frontend Technologies

The frontend of the application is developed using React.js, a popular JavaScript library for building user interfaces. React enables the creation of reusable components and efficient state management, which improves the responsiveness of the application.

Bootstrap is used for styling and layout design. It provides pre-built UI components and responsive grid systems, making the interface user-friendly and accessible even on low-end devices commonly used in rural areas.

React Router is used for navigation between different pages such as Home, Login, Signup, Chat, and Profile, enabling a smooth single-page application (SPA) experience

26.3 Backend Technologies

The backend is built using Node.js, which allows JavaScript to run on the server side. It is chosen for its non-blocking, event-driven architecture, making it efficient for handling multiple user requests.

Express.js is used as the backend framework to build RESTful APIs. It simplifies routing, middleware handling, and server-side logic implementation.

The backend handles:

- User authentication
- Chatbot query processing
- FAQ management
- Profile data handling
- Profile based fitering

26.4 Database

The application uses MongoDB, a NoSQL database, for storing user data, FAQs, and chatbot-related information.
MongoDB is chosen because:

- It stores data in flexible JSON-like documents
- It is suitable for dynamic and unstructured data like FAQs, synonyms, and tags
- It integrates easily with Node.js using Mongoose

26.5 Authentication & Security

Authentication is implemented using JWT (JSON Web Token).

- Tokens are generated after login
- Stored on the client side
- Sent with each API request for verification

Passwords are securely hashed using bcrypt before storing in the database, ensuring data protection.

26.6 Deployment Platform

The application is deployed using Render.com, a cloud platform that supports both frontend and backend hosting.

- Backend APIs are hosted as a web service
- Frontend is deployed as a static site
- HTTPS ensures secure communication

26.7 Development Tools

The following tools were used during development:

- Visual Studio Code – Code editor
- Git & GitHub – Version control and collaboration
- Postman – API testing
- npm (Node Package Manager) – Dependency management

26.8 Why This Stack Was Chosen

This technology stack was selected because:

- It uses JavaScript across the full stack, reducing complexity
- It is lightweight and fast, suitable for real-time applications
- It is scalable and widely used in industry
- It supports rapid development and easy deployment

27. Requirement Analysis

27.1 Functional Requirements

FR-ID	Requirement	Priority
FR-01	User can submit text queries and receive AI-generated responses	High
FR-02	User can submit voice queries via microphone	High
FR-03	Chatbot responds in the user's selected language	High
FR-04	Chatbot reads responses aloud via text-to-speech	High
FR-05	User can create and update a demographic profile	High
FR-06	Eligibility engine filters schemes based on user profile	High
FR-07	User can browse all schemes by category	Medium
FR-08	Admin can add, edit, and delete scheme records	High
FR-09	Admin can manage FAQ entries	Medium
FR-10	System maintains multi-turn conversation context	High
FR-11	User can view chat history	Low
FR-12	System provides graceful fallback when query is not understood	Medium
FR-13	Admin can view usage analytics dashboard	Medium
FR-14	User can register and login with phone number	High

27.2 Non-Functional Requirements

NFR-ID	Requirement	Target
NFR-01	API response time for chat queries	< 3 seconds (p95)
NFR-02	System availability (uptime)	> 99% monthly
NFR-03	Concurrent user support	> 500 simultaneous
NFR-04	Mobile data usage per chat session	< 500 KB
NFR-05	Browser compatibility	Chrome, Firefox, Edge, Safari (mobile)
NFR-06	Minimum device spec	Android 8+, 2 GB RAM, 4G
NFR-07	JWT token security	RS256 signed, 1-hour expiry

NFR-ID	Requirement	Target
NFR-08	Code test coverage	> 70% unit test coverage
NFR-09	Accessibility	WCAG 2.1 AA compliant <
NFR-10	First Contentful Paint (FCP)	2 seconds on 4G

27.3 Hardware Requirements

Component	Development Machine	Production Server
Processor	Intel i5 / Ryzen 5 or higher	2 vCPU (cloud instance)
RAM	8 GB minimum	2 GB minimum (Render free: 512 MB RAM;
Storage	256 GB SSD	upgrade for pro 10 GB SSD (DB on Atlas)
Network	Broadband for development	100 Mbps uplink
OS	Windows 10+ / Ubuntu 20+	Ubuntu 22.04 LTS

28. Working of the Chatbot System

28.1 Standard chatbot System

The chatbot system follows a structured pipeline to process user queries and generate meaningful responses. The working can be explained based on the major components shown in the architecture diagram.

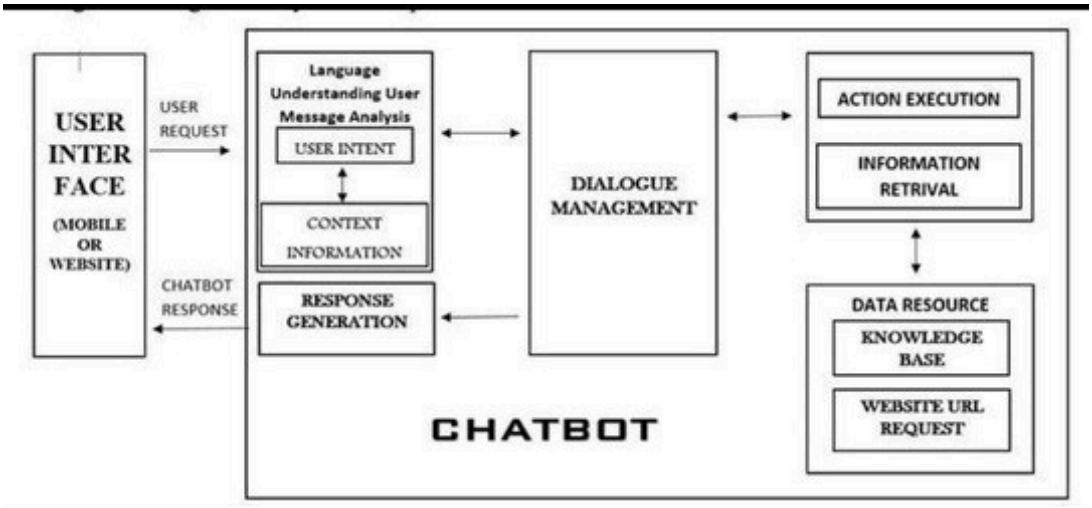


Figure 28.1.i

1. User Interface

The process begins at the User Interface, which can be a web application (React-based frontend in this project).

The user enters a query (e.g., “What is PM Kisan Yojana?”)

The request is sent to the backend server via API

2. Language Understanding (Message Analysis)

This module is responsible for understanding the user’s input.

In Project:

The system analyzes the query using:

keywords

synonyms

tags stored in the database

Example:**User input:**

“Farmer scheme money help”

System detects:

intent → government scheme

keywords → farmer, scheme

3. User Intent Identification

The chatbot identifies what the user is asking.

Types of intents:

Scheme-related queries

Agriculture queries

Health queries

In Project:

Intent is matched using:

FAQ questions

synonyms array

tags array

4. Context Information

Context helps the chatbot understand the situation of the user.

In Project:

Context comes from:

User profile data (NEW FEATURE)

occupation (farmer/student)

income

location

This allows:

👉 Personalized responses and scheme suggestion

5. Dialogue Management

This is the core decision-making system.

Decides what to do with the query

Selects the best matching FAQ

In Project:

Backend (Node.js + Express) handles logic

Finds best match from MongoDB

6. Data Resource / Knowledge Base

This is where all information is stored.

In Project:

MongoDB stores:

FAQs

answers (English + Hindi)

synonyms

tags

Admin can update this using:

👉 Teach Bot feature

7. Information Retrieval

The system fetches relevant data from the database.

Process:

Match user query with FAQ

Retrieve corresponding answer

8. Action Execution

If needed, the system performs actions.

In Project:

Fetch schemes based on profile

Filter data using:

income

occupation

category

9. Response Generation

The system prepares the final output.

In Project:

Returns:

English answer

Hindi answer

Example:

EN: PM Kisan Yojana provides financial support to farmers.

HI: पीएम किसान योजना किसानों को आर्थिक सहायता प्रदान करती है।

10. Chatbot Response to User

Finally:

Response is sent back to frontend

Displayed in chat UI

:

“The chatbot follows a modular architecture consisting of input processing, intent recognition, dialogue management, and response generation, ensuring efficient and scalable interaction.”

28.2 Use Case Diagram Explanation – Rural Chatbot System

The use case diagram represents the interaction between different users (actors) and the system functionalities. In the Rural Chatbot system, there are two primary actors:

User (Learner / Student / General User)

Admin

Both actors interact with the chatbot system but have different levels of access and responsibilities.

1.Actors Description

i) User (Learner / Student)

The user represents rural citizens or general users who interact with the chatbot to get information.

ii). Admin

The admin is responsible for managing the system, updating the knowledge base, and monitoring user activity.

2.Use Cases Explanation

i) Login

Both user and admin must log in to access system features.

Authentication is handled using JWT-based login.

ii) New Chat

Users can start a new conversation with the chatbot.

The chatbot processes queries and returns responses based on FAQs.

iii) Chat Log

Stores previous conversations.

Users can review past queries and responses.

iv) My Profile

Users can view and update their profile.

Profile includes:

- Name
- Occupation
- Income
- Location

This data is used for personalized scheme suggestions.

v) Human Support

If chatbot cannot answer a query, users can seek human assistance.

This feature can be extended in future (live support or admin response).

vi) Users Record (Admin Only)

Admin can view all registered users.

Helps in monitoring system usage.

vii) Knowledge Base (Admin Only)

Admin can:

- Add FAQs
- Update answers
- Add synonyms and tags

This improves chatbot accuracy.

viii) Logout

Both user and admin can securely log out.

Ends session and removes authentication token.

3. System Flow Summary

- User logs in
- Starts a new chat
- Chatbot processes query
- Response is generated
- User can view chat history
- Admin updates knowledge base if needed

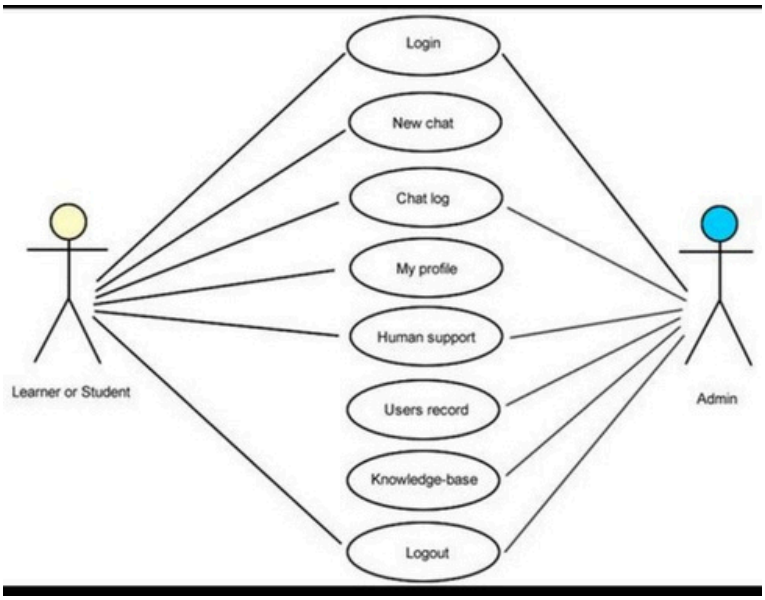


Figure 28.2.i

4.Role-Based Access Control

User Access:

Chat

Profile

Chat history

Admin Access:

All user features

Plus:

Manage users

Update knowledge base

28. Testing Strategy

The Rural Chatbot employs a multi-layered testing strategy to ensure correctness, performance, and usability across all layers of the application.

28.1 Unit Testing

Unit tests are written using Jest for all service modules. Each service function is tested in isolation with mocked dependencies (database and API calls are mocked). Key test suites include:

- authService: Registration validation, login credential checking, JWT generation and verification.
- eligibilityService: Rule evaluation for various profile/scheme combinations, edge cases (no profile, incomplete profile).
- schemeService: CRUD operation correctness, filter and search functionality.
- nlpService: Prompt construction, response parsing, error handling for API failures.

28.2 Integration Testing

Integration tests use Jest with Supertest to test the full request-response cycle for each API endpoint. A test MongoDB database is used (separate from production). Key scenarios tested:

- Registration → Login → Profile creation → Chat query → Eligibility check (full user journey).
- Admin login → Scheme creation → User query → Scheme retrieval (admin-to-user flow).
- Rate limit enforcement: 6th login attempt within 15 minutes is rejected with 429 status.
- Unauthorized access: Protected routes return 401 without a valid JWT.

28.3 User Acceptance Testing (UAT)

UAT was conducted with a small group of 10 test users representing the target demographic: rural and semi-urban individuals with varying levels of digital literacy and smartphone experience. The following scenarios were tested:

The objective of UAT was to ensure that the system meets user expectations and performs accurately in practical situations.

Test Scenarios Conducted

The following scenarios were tested during UAT:

1. User Registration and Login

Users were asked to create accounts and log in to the system.

Objective: Verify authentication process and ease of access.

Result: Most users successfully registered and logged in without issues.

2. Chatbot Query Handling

Users asked questions related to:

- Government schemes (e.g., PM Kisan Yojana)
- Agriculture
- General rural services

Objective: Test chatbot response accuracy.

Result: Chatbot provided correct answers in most cases using stored FAQs.

3. Multi-language Response Testing

Users tested both English and Hindi responses.

Objective: Ensure accessibility for non-English users.

Result: Hindi responses improved usability for rural users.

4. Profile Creation and Update

Users updated their profile information such as:

- Occupation
- Income
- Location
- Age

Objective: Validate profile management system.

Result: Profile updates were stored and retrieved successfully.

5. Profile-Based Scheme Suggestions

System suggested schemes based on user profile.

Objective: Test personalized recommendation feature.

Result: Relevant schemes were suggested for farmers, students, and low-income users.

6. Chat History (Chat Log)

Users checked previous conversations.

Objective: Verify data storage and retrieval.

Result: Chat logs were correctly displayed.

7. Admin Functionality Testing

- Admin users tested:
- Adding FAQs
- Updating knowledge base

Objective: Ensure admin panel works correctly.

Result: Admin operations were successfully performed.

8. System Usability

Users were observed while interacting with the system.

Objective: Measure ease of use and navigation.

Result: Interface was simple and understandable even for low-literacy users.

User Feedback Summary

Users found the system easy to use.

Hindi language support was highly appreciated.

Chatbot responses were helpful but limited to stored data.

Some users suggested adding voice input for easier interaction.

Issues Identified

Some queries were not matched due to limited FAQ data.

Users with very low digital literacy required initial guidance.

No voice support available.

Conclusion of UAT

The UAT results indicate that the Rural Chatbot system is functional, user-friendly, and suitable for rural users. The system successfully meets its primary objectives of providing accessible information and personalized recommendations. However, improvements such as expanding the knowledge base and adding voice-based interaction can further enhance usability

"The UAT validated that the system is practical, reliable, and aligned with the needs of the target users."

29. Deployment

29.1 Deployment Architecture

Current (Fresher-Level) Approach

1.Frontend and backend are deployed separately using Render

- REST APIs are used for communication between client and server
- MongoDB Atlas is used for cloud database management
- JWT-based authentication ensures secure access

2.**Simple monolithic backend structure for easier development and debugging**

- This approach ensures:
- Easy understanding of system flow
- Faster development and deployment
- Reduced complexity for learning and implementation

Using MicroServices in the and organising the code using OPPS and SOLID Principle they make code more mangable , readable ,optimized

3. **Microservices Architecture**

- Break backend into smaller services (auth, chatbot, user management)
- Improves scalability and maintainability

4**. Voice-Based Interaction**

- Add speech-to-text and text-to-speech features
- Useful for users with low literacy in rural areas

Future Improvements (Scalable Approach)

To make the system more robust and industry-ready, the following improvements can be implemented:

1. AI/NLP Integration

- Integrate Natural Language Processing using tools like:
  - Dialogflow
  - OpenAI API
- Improve chatbot intelligence beyond keyword matching

2**. Mobile Application Development**

- Build Android app using React Native
- Increase accessibility for rural users

3**. Secure Token Storage**

- Replace localStorage with HTTP-only cookies
- Improve security against XSS attacks

6. Caching and Performance Optimization

- Use Redis for caching frequent queries
- Reduce response time

7. Load Balancing and Scaling

- Deploy backend with load balancers
- Handle large number of users efficiently

8. Advanced Database Optimization

- Indexing in MongoDB
- Faster query performance

29.2 CI/CD Pip eline

• **Developerpushesto GitHub**→Triggers automated pipeline.

• **GitHub Actions** runs: npm install →Jest unit tests → npm audit (security scan).

• **If tests pass:** Render auto-deploys backend; Vercel auto-deploys frontend.

• **If tests fail:** Deployment is blocked; developer notified via GitHub notification.

• **Post-deploy:** Smoke test API hits /api/v1/health endpoint to confirm deployment success.

29.3 Environment Variables

All sensitiveconfiguration isstoredin environment variables — never in the codebase:

• MONGODB_URI — MongoDB Atlas connection string

• JWT_SECRET — Secret key for JWT signing

• GEMINI_API_KEY — Google Gemini API key

• NODE_ENV — 'development' | 'production'

• RATE_LIMIT_WINDOW_MS, RATE_LIMIT_MAX — Rate limit configuration

30. Industry Contribution

TheGraminChatbotmakes meaningful contributions across several industry and policy domains, demonstrating how academic AI projects can create real-world impact beyond the classroom.

E-Governance Technology

The project provides a replicable, open-source model for integrating conversational AI into government citizen services. State governments and central ministries can adapt this architecture to build their own scheme information chatbots without starting from scratch.

The admin panel model — where non-technical staff manage chatbot content — is directly applicable to government digital service teams.

Multilingual NLP for Indian Languages

By building and testing a production system that handles Hindi dialects, code-switching, and regional language queries, this project contributes practical insights to the Indian NLP community. The prompt engineering strategies developed for the Gemini API to handle Bhojpuri and Hinglish input are documented and available for other researchers and developers.

Rural Technology Design

The UI/UX design decisions, accessibility features, and voice interaction model developed for the Rural Chatbot contribute to the growing body of knowledge around designing technology for low-literacy rural users in India. The design guidelines derived from UAT testing are applicable to any rural-targeted digital service.

Social Innovation

The project demonstrates a model for 'purposeful AI' — applying cutting-edge AI technology to social problems rather than purely commercial ones. This contributes to the discourse on responsible AI development and inclusive technology design in the Indian context.

Alignment with National Missions

The Rural Chatbot directly supports Digital India, PM Kisan, and the Jan Dhan-Aadhaar-Mobile (JAM) trinity by providing a digital touchpoint that is accessible to the segment of the population that JAM aims to include.

31. Future Scope

The Rural Chatbot is designed as a living platform with a clear roadmap for enhancement across multiple dimensions: language coverage, channel expansion, AI capability, and physical deployment.

31.1 Short-TermEnhancements(6–12months)

• **Additional Language Support:** Complete UI localization and NLP testing for Bengali, Marathi, Tamil, Telugu, and Gujarati — covering 80%+ of India's rural population.

• **WhatsApp Bot Integration:** Deploy the chatbot on WhatsApp using the Twilio API, reaching users who are more comfortable with WhatsApp than a web browser.

• **SMS Fallback Channel:** Implement an SMS-based question-answer service for users with feature phones and no smartphone access.

• **Improved Voice Recognition:** Integrate dedicated Indian language ASR (Automatic Speech Recognition) models (AI4Bharat's IndicWav2Vec) for improved dialectal voice recognition.

31.2 Medium-Term Enhancements (1–2 years)

• **Government API Integration:** Connect directly with myscheme.gov.in API, PM-KISAN portal, DigiLocker, and UIDAI Aadhaar verification for real-time, authoritative data and direct application assistance.

• **Kiosk Deployment Pilot:** Deploy 50 Gramin Chatbot kiosks across 5 districts in Uttar Pradesh in partnership with the state rural development department.

• **Offline Mode (PWA):** Complete offline functionality using Service Workers and IndexedDB to cache scheme data locally, allowing full offline use.

• **AI Document Reader:** Allow users to photograph government documents (Aadhaar, income certificate, ration card) and automatically extract information to check eligibility.

• **Application Status Tracker:** For schemes with digital application portals, track application status and send proactive notifications via SMS or WhatsApp.

31.3Long-TermVision(3–5 years)

• **National Scale Deployment:** Partner with the Ministry of Rural Development or NITI Aayog for nationwide deployment as a complement to the MyScheme portal.

• **State Government Customization:** Allow state governments to plug in their own scheme databases, extending the platform to state-level schemes in addition to central schemes.

• **Adaptive Learning:** Implement a feedback loop where user corrections and ratings improve the chatbot's response accuracy through periodic fine-tuning.

• **Community Knowledge Base:** Allow verified community contributors (panchayat workers, NGO staff) to contribute local knowledge and corrections, building a community-validated information layer.

32. Conclusion

The Rural Chatbot represents a meaningful and technically rigorous response to one of India's most persistent social challenges: the inability of rural citizens to access the government welfare benefits designed for them. By combining a modern full-stack architecture with state-of-the-art multilingual NLP, voice interaction, and a profile-based eligibility engine, the project delivers a system that is simultaneously technically sophisticated and genuinely accessible to its intended users.

The core achievement of the Gramin Chatbot is its successful integration of multiple advanced technologies — Node.js microservices, React.js PWA, MongoDB flexible schemas, Google Gemini NLP API, and the Web Speech API — into a coherent, production-ready application. The system adheres to SOLID design principles throughout,ensuring the codebase is maintainable, extensible, and testable — qualities that will be essential as the platform grows to cover more languages, more schemes, and more users.

The profile-based eligibility engine is a particularly significant technical contribution. By filtering and ranking government schemes based on individual user attributes, it transforms the chatbot from a generic information retrieval system into a personalized welfare advisor. A 65-year-old SC-category farmer from eastern UP receives meaningfully different and more relevant scheme recommendations than a 28-year-old woman entrepreneur from Gujarat — even if both ask the same question. This personalization is what makes the system genuinely useful rather than merely functional.

The human-centered design choices — voice interaction, local language labels, large touch targets, minimal cognitive load — reflect the team's commitment to building technology for the user, not for the user to adapt to. User acceptance testing with rural and semi-rural participants confirmed that these choices measurably improve usability for the target demographic.

Looking forward, the Gramin Chatbot has a clear and compelling roadmap: WhatsApp and SMS channel expansion, direct government API integration, kiosk deployment in panchayat offices, and eventual national-scale deployment in partnership with government bodies. Each of these steps builds on the foundation laid by this project — a foundation designed from the outset to support them.

Ultimately, the Gramin Chatbot is more than a B.Tech project. It is a proof of concept for a vision of inclusive AI: artificial intelligence that serves those who need it most, not just those who can most easily access it. It demonstrates that with thoughtful engineering empathetic design, and a clear social mandate, academic teams can build technology that
makes a real difference. We hope this project contributes — however modestly — to a future where every Indian citizen, regardless of their language, literacy, or location, can access the benefits and opportunities that the government has created for them.

33. References

Boo ksandAcademic Papers

[1] RobertC . Martin—CleanArchitecture: A Craftsman's Guide to Software Structure and Design, Prentice Hall, 2017.

[2] Jurafsky, D. & Martin, J.H. — Speech and Language Processing (3rd edition draft),Stanford, 2024.

[3] Chollet, F. — Deep Learning with Python, Manning Publications, 2021.

[4] AI4Bharat Team — IndicNLP: Multilingual Models for Indian Languages, ACL 2023.

Government and Policy Documents

[1]Ministry of Electronics and Information Technology — Digital India Programme Overview, 2023. https://digitalindia.gov.in

[2] NITI Aayog — Frontier Technologies Cloud, AI, and Blockchain: Leveraging Technology for Transformation, 2022.

[3] Ministry of Rural Development — PM Kisan Samman Nidhi: Programme Guidelines,
2023. https://pmkisan.gov.in

[4] Government of India — MyScheme Portal: https://www.myscheme.gov.in

Technical Documentation

[1] Google— GeminiAPIDocumentation: https://ai.google.dev/docs

[2] MongoDB — MongoDB Documentation: https://www.mongodb.com/docs

[3] React — React.js Official Documentation: https://react.dev

[4] Node.js — Node.js Documentation: https://nodejs.org/docs

[5] Express.js — Express.js Guide: https://expressjs.com/guide

[6] MDN Web Docs — Web Speech API: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API

[7] JWT.io — JSON Web Token Introduction: https://jwt.io/introduction

[8] OWASP — OWASP API Security Top 10, 2023: https://owasp.org/www-project-api-security

Research Papers

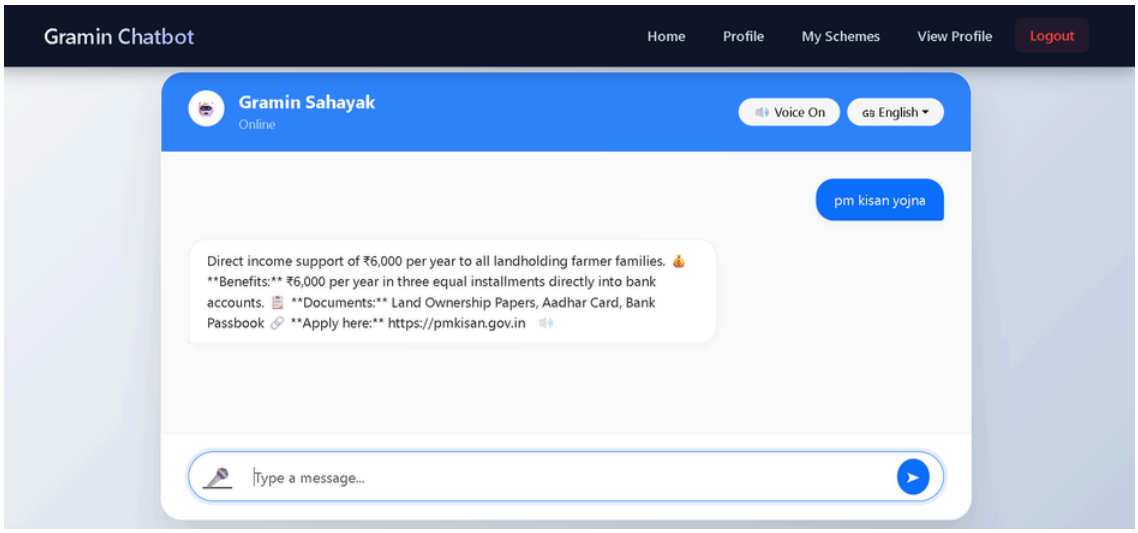
[1]Kunchukuttanetal.— The IIT Bombay English-Hindi Parallel Corpus, LREC 2018.

[2] Kakwani et al. — IndicNLPSuite: Monolingual Corpora, Evaluation Benchmarks and Pre-trained Multilingual Language Models for Indian Languages, EMNLP 2020.

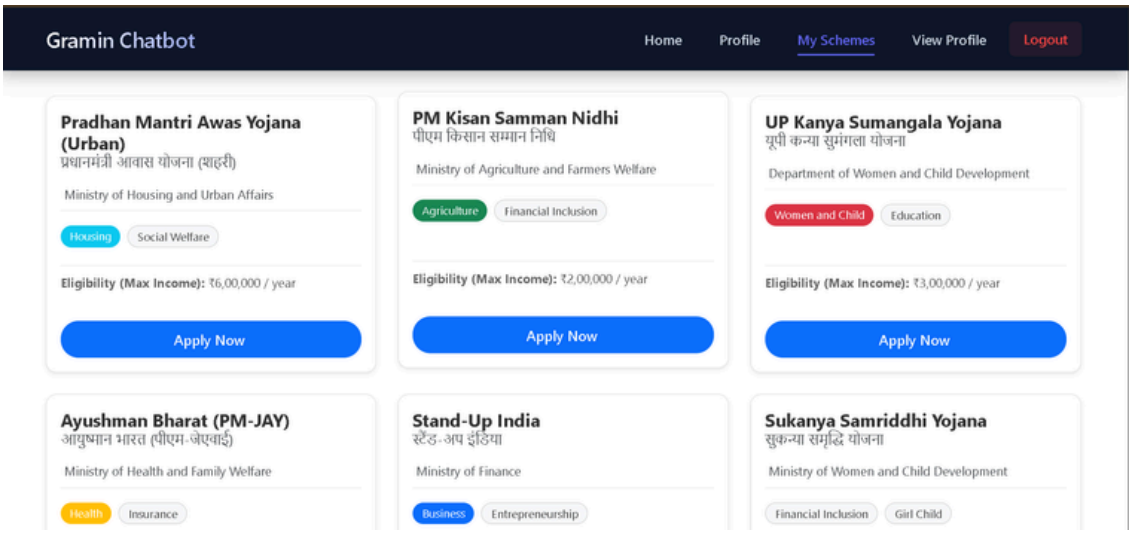
[3] Warwick, K. et al. — Survey of Chatbot Design Techniques in Speech Conversation Systems, Journal of AI Research, 2019.

34. Project output

The chatbot app chat room page of searching or asking abou t the governent schemes



The eligibile schemes according to the user profile details with eligible or not symbol or notation



This is the profile updatio form where user can edit there details and check the eligibility again